

Informe de Pruebas de Calidad y Control de Errores

NexaSafe — Estrategia de pruebas y estado inicial (iteración 0)

NEXASAFE

PROGRAMA

Ingeniería de Sistemas

EQUIPO

Juan Felipe Pineda Cardona · Integrante 2 · Integrante 3

VERSIÓN DEL DOCUMENTO

1.0 (iteración 0)

INSTITUCIÓN

Fundación Universitaria San Mateo

FECHA DE ENTREGA

22 de septiembre de 2026

PRÓXIMA ACTUALIZACIÓN

Cierre del Sprint 4 — hito H5

1. Introducción

Nota sobre el alcance de esta iteración

A la fecha de este informe (22 de septiembre de 2026) el proyecto se encuentra en el **Sprint 0**: el pipeline DevSecOps se está habilitando y ninguna historia de negocio de NexaSafe se ha construido todavía. El código existente en el repositorio corresponde a un prototipo previo (*SanMateoApp*) con un único flujo de autenticación local contra un JSON de prueba, descrito en docs/descripcion-aplicacion.md. Por lo tanto, este documento entrega la **estrategia y el plan de pruebas** que gobernará la calidad de NexaSafe durante el semestre, no resultados de ejecución de pruebas sobre funcionalidades que aún no existen. Se actualizará con resultados reales al cierre de cada sprint, y de forma consolidada en el hito H5 (25 de noviembre).

El objetivo de este informe es doble: (1) fijar la estrategia de pruebas y los criterios de calidad que todo incremento de NexaSafe debe cumplir para considerarse terminado, y (2) dejar registro trazable del estado de calidad del proyecto en cada punto de control del semestre, incluyendo esta primera iteración.

2. Estrategia de pruebas

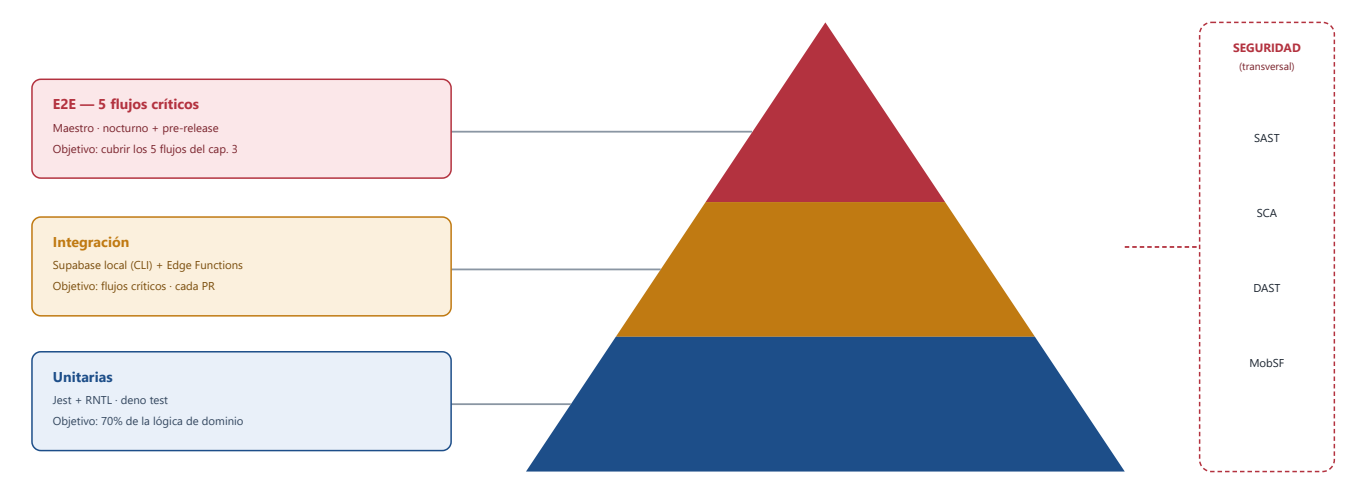


Figura 2.1 — Pirámide de pruebas de NexaSafe, con la seguridad como capa transversal a todos los niveles.

Nivel	Cobertura objetivo	Herramienta	Ejecución
Unitarias	70 % de lógica de dominio	Jest + RNTL, Deno test	Cada PR
Integración	Flujos críticos (alerta, trayecto, auth)	Supabase local vía CLI con base efímera; pruebas de Edge Functions y de políticas RLS	Cada PR
E2E	5 flujos críticos	Maestro	Nightly
Seguridad	SAST/SCA/DAST/MobSF	Semgrep, CodeQL, npm audit, OWASP ZAP, MobSF	Cada PR / nightly / release
Aceptación	Criterios Gherkin de cada historia	Manual en dispositivo físico	Antes de la review

3. Flujos E2E críticos

Estos cinco flujos son obligatorios y se ejecutan de forma automatizada con Maestro cada noche y antes de cada release, a partir de que las épicas correspondientes estén construidas.

#	Flujo
1	Registro de guardián → alta de menor → validación del colegio → primer login del protegido.
2	Inicio de trayecto → seguimiento → llegada → cierre automático.
3	Desvío del corredor → temporizador → sin respuesta → alerta automática. Trabajo futuro (E5)
4	Botón de pánico → cancelación exitosa con PIN.
5	Botón de pánico → alerta enviada → recepción del guardián → "voy en camino" → cierre clasificado.

4. Plan de pruebas de seguridad

Práctica	Herramienta	Cuándo se ejecuta
Análisis estático de código (SAST)	Semgrep + CodeQL	En cada PR
Análisis de composición de software (SCA)	npm audit, Dependabot	En cada PR + semanal
Escaneo de secretos	Gitleaks	Local (pre-commit) y en cada PR
Análisis dinámico de la API (DAST)	OWASP ZAP baseline	Nightly contra staging
Análisis del binario móvil	MobSF sobre el APK	Pre-release
Verificación de políticas RLS	Supabase CLI + suite de pruebas de política	Antes de cada despliegue

4.1 Quality gate del pipeline

Severidad	SLA de corrección	Efecto
CRITICAL	24 h	Bloquea el merge y el despliegue
HIGH	Dentro del sprint actual	Bloquea el merge
MEDIUM	Siguiente sprint	Advertencia; entra al backlog como issue security
LOW	Backlog priorizado	Registrado, sin bloqueo

Las etapas de secretos, SAST, SCA y verificación de políticas RLS son **bloqueantes** ante severidad High o superior: un PR que falle cualquiera de ellas no puede fusionarse, sin importar la urgencia (ver `PLAN_NEXASAFE.md`, sección 10.2).

5. Criterios de aceptación de calidad

Funcional

- Todos los criterios de aceptación en Gherkin verificados
- Probada en dispositivo Android físico, no solo en emulador

Calidad de código

- Pull Request revisado y aprobado por al menos un par
- Cobertura de pruebas unitarias $\geq 70\%$ en el módulo tocado
- Lint y formateo sin errores (ESLint + Prettier)
- Sin TODO ni código comentado en la rama principal

Seguridad — obligatorio, no negociable

- SAST sin hallazgos *High* o *Critical*
- SCA sin vulnerabilidades *Critical* explotables
- Escaneo de secretos limpio
- Ningún dato personal en logs (verificado manualmente en el PR)
- Tablas y Edge Functions nuevas con RLS, autorización por rol y rate limiting

6. Métricas de calidad objetivo

6.1 Métricas de seguridad

Métrica	Objetivo
Vulnerabilidades Critical/High abiertas al cierre del sprint	0
Tiempo medio de remediación de hallazgos High	< 5 días
Cobertura del pipeline de seguridad	100 % de los PR
Secretos filtrados detectados en main	0

6.2 Métricas de producto

Métrica	Objetivo
Latencia disparo → notificación recibida	< 5 s con red, < 30 s con reconexión
Precisión de posición reportada	< 20 m en zona urbana abierta
Consumo de batería por hora de trayecto	< 8 %
Tasa de falsas alarmas automáticas	< 10 % de los trayectos

7. Plan de prueba de campo

En el Sprint 6 (ampliación posterior al cierre del semestre, fuera de la ruta crítica actual) se ejecuta una prueba real de trayecto en un recorrido de aproximadamente 1 km, midiendo: precisión del GPS, consumo de batería por hora de trayecto, latencia entre disparo y notificación recibida, y comportamiento en zonas de baja cobertura. Los resultados retroalimentan directamente las métricas de producto del capítulo 6.

8. Gestión de hallazgos y control de errores

8.1 Proceso de triage

1. El pipeline reporta hallazgos automáticamente en cada PR (SAST, SCA, secretos, políticas RLS).
2. El Security Champion del sprint tría los hallazgos antes de la Security Review (30 min, previa a la Sprint Review).
3. Los hallazgos **CRITICAL** y **HIGH** bloquean el merge hasta corregirse; los de severidad **MEDIUM** o **LOW** entran al Product Backlog como issue security.
4. Toda excepción se documenta en `/docs/security/exceptions.md` con justificación, responsable y fecha de vencimiento. No hay excepciones permanentes.

8.2 Control de errores funcionales

Todo defecto funcional detectado en pruebas manuales, E2E o en la Sprint Review se registra como issue de tipo bug en GitHub, con pasos de reproducción, resultado esperado vs. obtenido y severidad. Los defectos que bloquean un criterio de aceptación de una historia en curso se corrigen antes de dar por cumplida la Definition of Done; los defectos sobre funcionalidad ya cerrada entran priorizados al Product Backlog.

9. Estado actual y próximos pasos

Elemento	Estado	Detalle
Pipeline de CI con lint + test + build	EN CURSO	Objetivo del Sprint 0 (historia E0-02); se verifica en el hito H1 (2 oct).
SAST / SCA / secretos automatizados	EN CURSO	Historia E0-03, Sprint 0.
Funcionalidad NexaSafe construida	SIN INICIAR	Ninguna épica de negocio (E1–E11) se ha construido; el repositorio solo contiene el prototipo <i>SanMateoApp</i> con login local.
Cobertura de pruebas unitarias	NO APLICA AÚN	Se medirá desde el primer PR de una historia de negocio (Sprint 1).
Hallazgos Critical/High abiertos	0	No hay hallazgos porque aún no hay pipeline ejecutándose sobre código de producto.

Qué se puede probar hoy

El único flujo ejecutable actualmente es el login local del prototipo *SanMateoApp* (AuthScreen → useAuthViewModel → AuthenticateUser → LocalAuthRepository), documentado en docs/descripcion-aplicacion.md. Ese flujo se usará como caso de prueba piloto para validar la configuración de Jest y React Native Testing Library antes de que exista la primera historia de negocio de NexaSafe.

Este informe se actualizará al cierre de cada sprint con resultados reales de cobertura, hallazgos de seguridad y defectos abiertos/cerrados, y de forma consolidada en el hito H5 (25 de noviembre) como el "reporte consolidado de seguridad" declarado en los entregables académicos.

Anexo Plantilla de reporte de incidencia

Título: resumen corto y específico del defecto.

Historia / épica relacionada: ID del backlog.

Severidad: Critical / High / Medium / Low.

Pasos de reproducción: lista numerada.

Resultado esperado: según el criterio de aceptación.

Resultado obtenido: comportamiento observado.

Evidencia: captura, log o video.

Entorno: dispositivo, versión de la app, ambiente (dev/staging).